DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY FACULTY OF TECHNOLOGY

UNIVERSITY OF RUHUNA

DATABASE MANAGEMENT SYSTEMS PRACTICUM

ICT 1222

ASSIGNMENT 01 – MINI PROJECT

(Student Details, Attendance, Marks, and Result Management)

GROUP 03

SUBMITTED TO: Ms.Sandaruwani pathirage

SUBMITTED BY:

TG/2024/2127 – D.S.R.N.Dahanayaka

TG/2024/2088 – T.H.D wosada

TG/2024/2086 – K.A.M.I.Geethanjana

TG/2024/2094 – H.A.I. Nimsara

DATE OF SUBMISSION: 08 th May of 2026

INTRODUCTION

1.1 Brief introduction about the problem The Faculty of Technology at the University of Ruhuna manages hundreds of students across various course units. Currently, the process of recording student details, tracking attendance across a 15-week semester, and calculating Continuous Assessment (CA) and final exam marks is a complex administrative task.

The primary challenges identified include:

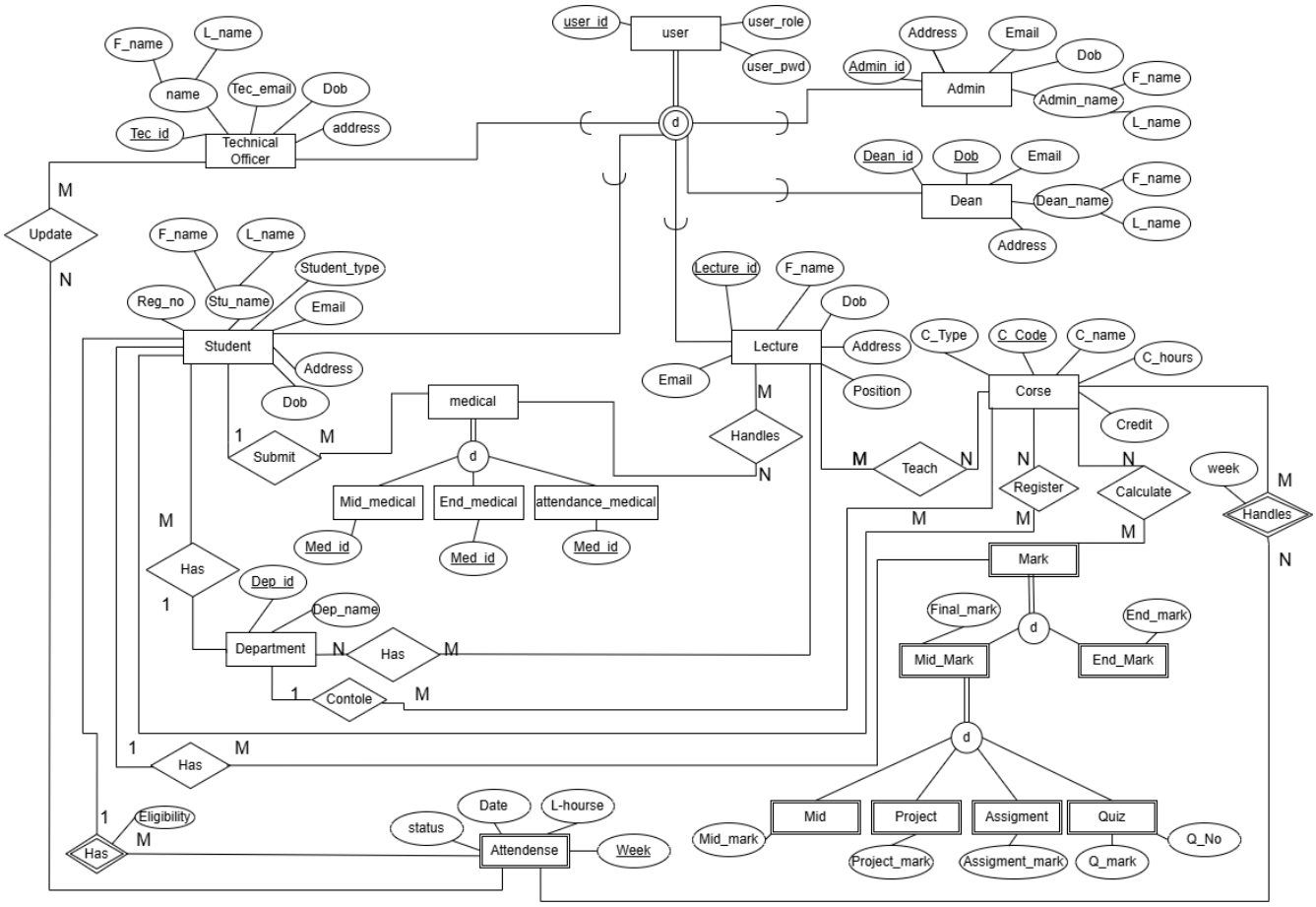
- Attendance Tracking: Calculating the 80% eligibility threshold manually for both theory and practical sessions is prone to human error.
- Special Cases: Managing medical certificates (MC) for missed sessions or exams requires precise adjustments to eligibility and result calculations.
- Complex Grading Rules: Implementing specific regulations from UGC Commission Circular No. 12-2024, such as the grade cap for repeat students (maximum 'C' grade) and withholding results (WH) for suspended students, is difficult to maintain in traditional spreadsheets.
- Data Security: Sensitive student results must only be accessible to authorized personnel (Admin, Dean, Lecturers) while students should only have read-only access to their own data.

1.2 Brief introduction to the solution Our solution is a comprehensive "Faculty Management System" built on a Relational Database Management System (RDBMS) using MySQL. This system provides a centralized repository for all student-related data. Key features include:

- Automated Eligibility Logic: Uses SQL Views to instantly calculate attendance percentages and mark students as "Eligible" or "Not Eligible" for final exams.
- Role-Based Access Control: Implements high-level security through MySQL user accounts with granular privileges (GRANT/REVOKE).
- Standardized Grading: A robust grading engine implemented via Views that automatically handles MC, WH, and repeat student caps as per University guidelines.
- Batch and Individual Reporting: Stored procedures that allow the Dean and Lecturers to view summaries of an entire batch or search for specific individual records instantly.

DATABASE DESIGN

2.1 Proposed ER/EER Diagram The Enhanced Entity-Relationship (EER) diagram for the Faculty Management System illustrates the complex relationships between students, academic staff, and administrative entities. A key feature of this design is the specialization of the “User” entity into five distinct roles: Admin, Dean, Lecturer, Student, and Technical Officer. This ensures a secure login system where each person is linked to their professional or academic profile.



Description of Entities and Relationships:

- User Supertype: The central entity that stores credentials. It has a one-to-one relationship with specific role tables (Student, Lecturer, etc.).
- Student and Course (Many-to-Many): Managed via the "Register" table. A student registers for many courses, and a course has many students.
- Attendance (Weak Entity): Connected to Student, Course, and Technical Officer. It tracks 15 weeks of data, categorizing sessions into Theory and Practical.
- Mark Entity: Stores all assessment components. It is linked to the Student and Course to ensure that marks are unique to each registration.
- Medical Entity: A specialized table that records medical justifications. It is linked to Students and handled by Lecturers.

2.2 Relational Mapping Diagram The following mapping translates the EER diagram into a relational schema. Primary Keys (PK) are underlined, and Foreign Keys (FK) are marked with an asterisk (*).

1. Department (dep_id, dep_name)
2. Course (c_code, c_name, credit, c_type)
3. Student (reg_no, f_name, l_name, email, address, dob, student_type, dep_id*, cgpa)
4. Admin (admin_id, f_name, l_name, email, dob, address)
5. Dean (dean_id, f_name, l_name, email, dob, address)
6. Lecturer (lecture_id, f_name, l_name, email, dob, address, position, dep_id*)
7. Technical_Officer (tec_id, f_name, l_name, email, dob, address)
8. Register (reg_no*, c_code*)
9. Teach (lecture_id*, c_code*)
10. Attendance (att_id, reg_no*, c_code*, att_date, week_no, session_type,

status, tec_id*)

11. Mark (mark_id, reg_no*, c_code*, quiz, assessment, mid_theory, mid_practical, final_theory, final_practical, ca_total, has_medical_ca, has_medical_mid, has_medical_final)
12. Medical (med_id, reg_no*, c_code*, med_date, reason, exam_type)
13. User (user_id, user_role, user_pwd, ref_id)

TABLE STRUCTURE

3.1 Table: Student

This table stores the primary records for all students within the faculty. It includes a classification field to distinguish between proper, repeat, and suspended students.

Attribute	Data Type	Constraints	Description
reg_no	VARCHAR(20)	PRIMARY KEY	Unique student registration number.
f_name	VARCHAR(50)	NOT NULL	First name of the student.
l_name	VARCHAR(50)	NOT NULL	Last name of the student.
email	VARCHAR(100)	UNIQUE, NOT NULL	Official university email address.
address	VARCHAR(255)	-	Residential address.
dob	DATE	-	Date of Birth.
student_type	ENUM	NOT NULL	Proper, Repeat, or Suspended status.
dep_id	INT	FOREIGN KEY	Links to the Department table.
cgpa	DECIMAL(4,2)	DEFAULT NULL	Cumulative Grade Point Average.

3.2 Table: Course

Stores information regarding the course units offered by the ICT department and other departments.

Attribute	Data Type	Constraints	Description
c_code	VARCHAR(20)	PRIMARY KEY	Unique course code (e.g., ICT1205).
c_name	VARCHAR(150)	NOT NULL	Full name of the course unit.
credit	DECIMAL(3,1)	NOT NULL	Number of credits (e.g., 2.0, 3.0).
c_type	ENUM	NOT NULL	Theory, Practical, or Both.

3.3 Table: Department

A lookup table for the various departments within the Faculty of Technology.

Attribute	Data Type	Constraints	Description
dep_id	INT	PK, AUTO_INC	Unique ID for the department.
dep_name	VARCHAR(100)	NOT NULL	Name of the department.

3.4 Table: Register

A junction table representing the many-to-many relationship between Students and Courses.

Attribute	Data Type	Constraints	Description
reg_no	VARCHAR(20)	PK, FK	Link to Student.
c_code	VARCHAR(20)	PK, FK	Link to Course.

3.5 Table: Lecturer

Stores details of academic staff responsible for teaching and medical handling.

Attribute	Data Type	Constraints	Description
lecture_id	INT	PK, AUTO_INC	Unique identifier for lecturers.
f_name	VARCHAR(50)	NOT NULL	First name/Title.
l_name	VARCHAR(50)	NOT NULL	Last name.
email	VARCHAR(100)	UNIQUE, NOT NULL	University staff email.
position	VARCHAR(100)	-	Job title (e.g., Senior Lecturer).
dep_id	INT	FK	Department the lecturer belongs to.

3.6 Table: Technical_Officer

Stores details of staff members responsible for updating attendance records.

Attribute	Data Type	Constraints	Description
tec_id	INT	PK, AUTO_INC	Unique identifier for TOs.
f_name	VARCHAR(50)	NOT NULL	First name.
l_name	VARCHAR(50)	NOT NULL	Last name.
email	VARCHAR(100)	UNIQUE	Contact email.

3.7 Table: User

The central security table used for system authentication.

Attribute	Data Type	Constraints	Description
user_id	INT	PK, AUTO_INC	Internal user ID.
user_role	ENUM	NOT NULL	Admin, Dean, Lecturer, TO, Student.
user_pwd	VARCHAR(255)	NOT NULL	SHA2-256 hashed password.
ref_id	VARCHAR(20)	NOT NULL	Links to the specific role table ID.

3.8 Table: Teach

A junction table mapping lecturers to the specific courses they are in charge of.

Attribute	Data Type	Constraints	Description
lecture_id	INT	PK, FK	Link to Lecturer.
c_code	VARCHAR(20)	PK, FK	Link to Course.

3.9 Table: Attendance

This table records session-by-session attendance for every student over the 15-week semester.

Attribute	Data Type	Constraints	Description
att_id	INT	PK, AUTO_INC	Unique record ID.
reg_no	VARCHAR(20)	FK	Student registration number.
c_code	VARCHAR(20)	FK	Course being attended.
att_date	DATE	NOT NULL	Date of the session.
week_no	TINYINT	1 to 15	The specific academic week.
session_type	ENUM	NOT NULL	Theory or Practical.
status	ENUM	DEFAULT 'Absent'	Present, Absent, or Medical.
tec_id	INT	FK	The TO who recorded this entry.

3.10 Table: Mark

Stores all assessment data. It features a generated column to ensure the CA total is always correct based on its components.

Attribute	Data Type	Constraints	Description
mark_id	INT	PK, AUTO_INC	Record identifier.
reg_no	VARCHAR(20)	FK	Student ID.
c_code	VARCHAR(20)	FK	Course ID.
quiz	DECIMAL(5,2)	-	Marks for quizzes.
assessment	DECIMAL(5,2)	-	Marks for assessments.
mid_theory	DECIMAL(5,2)	-	Mid-semester theory marks.
mid_practical	DECIMAL(5,2)	-	Mid-semester practical marks.
ca_total	DECIMAL(5,2)	GENERATED	Auto-sum of Quiz+Ass+Mid components.
has_medical_ca	TINYINT(1)	-	Boolean flag for CA medicals.
has_medical_final	TINYINT(1)	-	Boolean flag for Final exam medicals.

3.11 Table: Medical

Records specific details regarding medical submissions.

Attribute	Data Type	Constraints	Description
med_id	INT	PK, AUTO_INC	Medical record ID.
reg_no	VARCHAR(20)	FK	Student who submitted.
med_date	DATE	NOT NULL	Date of the medical excuse.
exam_type	ENUM	-	Attendance, CA, Mid, or Final.
reason	VARCHAR(255)	-	Description of the medical condition.

SOLUTION ARCHITECTURE & ATTENDANCE LOGIC

The system follows a three-tier database architecture. The Data Storage Layer consists of normalized relational tables. The Logic Layer utilizes Views and Stored Procedures to perform real-time calculations. The Security Layer manages user access through granular MySQL privilege grants.

Tools and Technologies

- **Database Management System:** MySQL Server 8.0.
- **Database Design Tool:** MySQL Workbench (for EER modeling and Forward Engineering).
- **Security Implementation:** SHA2 (Secure Hash Algorithm) with 256-bit encryption for user passwords.
- **Documentation:** SQL Script files organized by functional modules (Procedures, Views, Users).

SYSTEM IMPLEMENTATION (LOGIC)

Attendance Tracking Logic

The system tracks attendance for 15 weeks. Per the requirements, a student must maintain a minimum of 80% attendance to be eligible for the final exam. Crucially, sessions marked as 'Medical' are counted as "Attended" for eligibility purposes.

The view vm_attendance_summary automates this calculation:

CREATE OR REPLACE VIEW vm_attendance_summary AS

```
SELECT
  a.reg_no,
  CONCAT(s.f_name, ' ', s.l_name) AS student_name,
  a.c_code,
  a.session_type,
  COUNT(*) AS total_sessions,
  SUM(CASE WHEN a.status IN ('Present', 'Medical') THEN 1 ELSE 0 END) AS attended,
  ROUND((SUM(CASE WHEN a.status IN ('Present', 'Medical') THEN 1 ELSE 0 END) / COUNT(*)) *
100, 2) AS attendance_pct,
  CASE
    WHEN ROUND((SUM(CASE WHEN a.status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(*)) * 100, 2) >= 80
    THEN 'Eligible'
    ELSE 'Not Eligible'
  END AS eligibility
FROM Attendance a
JOIN Student s ON s.reg_no = a.reg_no
GROUP BY a.reg_no, a.c_code, a.session_type;
```

GRADING LOGIC (UGC CIRCULAR 12-2024)

Final Grading Implementation

The core of the system is the grading engine, which adheres strictly to UGC Commission Circular No. 12-2024. This logic is implemented in the vw_final_results view.

The engine handles three critical exceptions:

1. Suspended Students: Any student with the status 'Suspended' has their result automatically displayed as 'WH' (Withheld).
2. Medicals: If a student has submitted a medical for the final exam (has_medical_final = 1), the result is displayed as 'MC' (Medical Certificate).
3. Repeat Students: As per university policy, the maximum grade a repeat student can achieve is a 'C'. The logic caps their mark at 50% and assigns the 'C' grade regardless of a higher raw score.

```
CREATE OR REPLACE VIEW vw_final_results AS
SELECT
  m.reg_no, m.c_code,
  CASE
    WHEN s.student_type = 'Suspended'   THEN 'WH'
    WHEN m.has_medical_final = 1       THEN 'MC'
    WHEN s.student_type = 'Repeat' AND (m.final_theory + m.final_practical)/2 > 50
       THEN 50   -- Mark capped at 50 for Repeats
    ELSE ROUND((m.final_theory + m.final_practical)/2, 2)
  END AS final_mark,
  CASE
    WHEN s.student_type = 'Suspended'   THEN 'WH'
    WHEN m.has_medical_final = 1       THEN 'MC'
    WHEN s.student_type = 'Repeat'     THEN 'C'
    WHEN (m.final_theory+m.final_practical)/2 >= 85 THEN 'A+'
    WHEN (m.final_theory+m.final_practical)/2 >= 75 THEN 'A'
    WHEN (m.final_theory+m.final_practical)/2 >= 65 THEN 'B+'
    WHEN (m.final_theory+m.final_practical)/2 >= 55 THEN 'B'
    WHEN (m.final_theory+m.final_practical)/2 >= 45 THEN 'C'
    ELSE 'F'
  END AS grade
FROM Mark m
JOIN Student s ON s.reg_no = m.reg_no;
```

REPORTING PROCEDURES

STORED PROCEDURES AND VIEWS

Stored procedures were developed to provide dynamic reporting for the Dean and Lecturers. These procedures use input parameters to filter data, providing a more user-friendly interface than raw SQL queries.

SGPA and Batch Summary

The system calculates the Semester Grade Point Average (SGPA) by converting alphabetical grades into grade points (e.g., A=4.0, B=3.0) and weighting them by course credits.

Cross-Tabulation (Pivot) Results

To provide a complete result sheet for the faculty, the sp_student_grade_final procedure was created. This procedure uses a pivot logic to show all course grades for a student on a single horizontal row, alongside their SGPA and calculated CGPA.

```
CREATE PROCEDURE sp_student_grade_final()
BEGIN
  SELECT
    s.reg_no AS `Index Number`,
    CONCAT(s.f_name, ' ', s.l_name) AS `Student Name`,
    MAX(CASE WHEN fr.c_code = 'ICT1201' THEN fr.grade END) AS `ICT1201`,
    MAX(CASE WHEN fr.c_code = 'ICT1205' THEN fr.grade END) AS `ICT1205`,
    MAX(CASE WHEN fr.c_code = 'MAT1201' THEN fr.grade END) AS `MAT1201`,
    ROUND(COALESCE(sg.SGPA, 0.00), 2) AS `SGPA`,
    ROUND((COALESCE(s.cgpa, 0) + COALESCE(sg.SGPA, 0)) / 2, 2) AS `CGPA`
  FROM Student s
  LEFT JOIN vw_final_results fr ON fr.reg_no = s.reg_no
  LEFT JOIN vw_sgpa sg ON sg.reg_no = s.reg_no
  GROUP BY s.reg_no, s.f_name, s.l_name, s.cgpa, sg.SGPA;
END$$
```

This procedure enables the administration to generate an official result transcript for the entire batch with a single command: CALL sp_student_grade_final();.

USER MANAGEMENT AND SECURITY

User Accounts and Privileges

The system implements a multi-user environment where data security is maintained through

Role-Based Access Control (RBAC). We created five distinct database accounts, each with specific privileges defined by their administrative or academic requirements.

1. Admin Account (lms_admin)

- Reason for creation: To have full oversight of the database, perform maintenance, and manage other user accounts.
- Privileges: Granted all privileges on all tables with the GRANT OPTION, allowing the admin to delegate permissions to others.
- Implementation:

codeSQL

CREATE USER 'lms_admin'@'localhost' IDENTIFIED BY 'Admin@1234!';

GRANT ALL PRIVILEGES ON university_db.* TO 'lms_admin'@'localhost' WITH GRANT OPTION;

2. Dean Account (lms_dean)

- Reason for creation: The Dean needs to oversee all data (marks, attendance, and student profiles) for decision-making but does not require the authority to change the database schema or manage other users.
- Privileges: Granted all CRUD (Create, Read, Update, Delete) privileges on all tables without the Grant Option.
- Implementation:

CREATE USER 'lms_dean'@'localhost' IDENTIFIED BY 'Dean@1234!';

GRANT ALL PRIVILEGES ON university_db.* TO 'lms_dean'@'localhost';

3. Lecturer Accou nt (lms_lecturer)

- **Reason for creation:** Lecturers are responsible for the entry and adjustment of marks and handling medical justifications for their respective courses.
- **Privileges:** Selective privileges to modify data across tables but restricted from administrative tasks.
- **Implementation:** CREATE USER 'lms_lecturer'@'localhost' IDENTIFIED BY 'Lecturer@1234!';
- GRANT SELECT, INSERT, UPDATE, DELETE ON university_db.* TO 'lms_lecturer'@'localhost';

4. Technical Officer Account (lms_to)

- **Reason for creation:** Technical Officers (TOs) are primarily responsible for recording daily attendance. To protect sensitive exam marks, they are restricted from viewing or editing any tables except those related to attendance.

- **Privileges:** Read and Write access to the Attendance table and Read access to the attendance summary view.
- **Implementation:**

```
CREATE USER 'lms_to'@'localhost' IDENTIFIED BY 'TechOff@1234!';

GRANT SELECT, INSERT, UPDATE ON university_db.Attendance TO 'lms_to'@'localhost';

GRANT SELECT ON university_db.vw_attendance_summary TO 'lms_to'@'localhost';
```

5. Student Account (lms_student)

- **Reason for creation:** Students need to check their results and eligibility status but must never be allowed to modify any data.
- **Privileges:** Strictly Read-Only (SELECT) access to specific **Views** only. They cannot see raw tables.
- **Implementation:**

```
CREATE USER 'lms_student'@'localhost' IDENTIFIED BY 'Student@1234!';

GRANT SELECT ON university_db.vw_final_results TO 'lms_student'@'localhost';

GRANT SELECT ON university_db.vw_attendance_summary TO 'lms_student'@'localhost';
```

Security Measures

Beyond privilege control, the system employs the following measures:

- **Password Hashing:** As seen in the User.sql script, passwords are not stored in plaintext. We utilize the SHA2 function to hash passwords, ensuring that even if the User table is compromised, actual passwords remain protected.
- **View-Based Access:** Students and Technical Officers are granted access to **Views** rather than raw tables. This abstraction hides sensitive columns (like internal admin notes or ID flags) and prevents direct unauthorized manipulation of records.

PROJECT MANAGEMENT AND HOSTING

8. PROJECT MANAGEMENT

8.1 Problems Faced

During the development of the Faculty Management System, several technical challenges were encountered:

1. **Complexity of Grading Rules:** Integrating the UGC Circular requirements (MC/WH status and Repeat caps) into a single query was difficult, as a student could simultaneously be a "Repeat" student but also have an "MC" for a specific exam.
2. **Attendance Aggregation:** Courses that offered both Theory and Practical sessions required a way to track attendance separately while still providing a combined percentage for final eligibility.

8.2 Solutions Applied

1. **Nested Case Logic:** We utilized nested CASE statements within SQL Views. By establishing a priority (e.g., 'Suspended' status takes priority over any calculated mark), we ensured the grading engine followed university policy perfectly.
2. **View Layering:** We created sub-views for "Theory Only" and "Practical Only" attendance, then joined them into a master "Eligibility View" to provide the required combined summary.

---

9. HOSTING AND DEPLOYMENT

9.1 Hosting Selection

We chose to host the backend on a **Local Server (Localhost)** using the MySQL engine provided by XAMPP.

- **Reason for Selection:** This approach provides a zero-cost environment for internal testing and rapid development. It allows for high-speed data manipulation without being affected by internet latency.

9.2 Cloud Environment Considerations

If the faculty chooses to host this backend in a cloud environment (e.g., AWS RDS, Azure SQL, or Google Cloud SQL), the following changes would be necessary:

1. **User Host Configuration:** MySQL users currently tied to 'localhost' must be updated to '%' or specific IP ranges to allow remote connections.
2. **Encryption in Transit:** Enabling SSL/TLS connections to ensure that student marks are not intercepted while being sent from the client to the cloud.
3. **Firewall Rules:** Configuring Security Groups to restrict access only to University IP addresses on port 3306.

INDIVIDUAL CONTRIBUTION

INDIVIDUAL CONTRIBUTION TO BACKEND DEVELOPMENT

The development of the system was divided into four functional modules. Each member was responsible for the full lifecycle of their module, including Table creation, Data insertion, View implementation, and User privilege configuration.

Member Index	Module Name	Specific Files & Contributions
TG2088	Foundation & Admin	Developed foundational schema (`create_database`, `Department`, `Course`). Managed `Admin` profiles and `Admin_user` privileges. Implemented the Attendance Summary View.
TG2086	Academics & Staff	Responsible for Staff modules (`Dean`, `Technical_Officer`, `Lecturer`). Implemented the `Teach` junction table and views for `Theory_only_attendance` and `Ca_mark_with_grading`.
TG2127	Student & Registration	Developed the `Student`, `User`, and `Register` modules. Implemented the core Grading Engine View (`Final_result_with_grade.sql`) and managed Lecturer/Student account security.
TG2094	Core Engine (Marks)	Developed the most data-intensive modules: `Mark`, `Attendance`, and `Medical`. Authored complex logic for `Eligibility`, `SGPA`, and all Stored Procedures for the system.

Collaborative Integration:
While members worked on individual files, the final integration was done through a shared Git repository. This allowed us to merge 25 distinct scripts into a single cohesive database system while ensuring that all Foreign Key cascades and View dependencies were maintained.

CONCLUSION AND REFERENCES

CONCLUSION

The Faculty Management System successfully integrates student management, attendance tracking, and result processing into a centralized MySQL database. By following a modular development approach, our team was able to build a robust system containing 13 tables, 6 complex views, and 11 stored procedures.

The system effectively automates the enforcement of UGC Commission Circular No. 12-2024, ensuring that repeat students are capped at a 'C' grade and suspended students have their results withheld ('WH'). The implementation of a strict 22-step execution order ensures that the database can be deployed reliably in any MySQL environment. Ultimately, this project demonstrates how structured relational design can simplify complex academic administration while maintaining high standards of data security and integrity.

REFERENCES

1. University Grants Commission (UGC): Commission Circular No. 12-2024: Guidelines for Examination Grading and Student Eligibility.
2. University of Ruhuna, Faculty of Technology: Level 01 Semester 02 Academic Timetable (ICT Department).
3. Oracle MySQL Documentation: MySQL 8.0 Reference Manual for DDL and DCL commands.
4. Database Design Principles: Elmasri, R., & Navathe, S. B. - Fundamentals of Database Systems.
5. Project Specification: ICT1222 - Database Management Systems Practicum Mini Project Guidelines.